



United International University
Department of CSE

Course Code: CSE 426/CSE 4326

Course Name: Microprocessor, Microcontroller and Interfacing Lab

Experiment no: 2

Experiment Name: : Wifi Communication and building IoT-based systems
Using XAMPP and ESP32

Submitted by:

Name: Mrs. Sk. Tonni Islam

Student ID: 011 221 329

Section: H

Date of Performance: July 21, 2026

Date of Submission: July 28, 2026

Objective:

The objective of this lab experiment is to set up an ESP32 board to build IoT-based systems. We Shall divide this experiment in the following manners:

- We will monitor temperature and humidity data using a DHT11 sensor that will be connected to ESP32. Then, the real time data shall be sent to a local server, called “XAMPP” where this data will be stored in a database. Additionally, we will display the data on a local web page. Students shall perform similar gas data monitoring using this “XAMPP” server and interface led buttons to control using the IoT cloud.
- Next, students will monitor the gas sensor data and control an LED using the Arduino Cloud IoT platform. This comprehensive exercise encompasses multiple key steps, starting with the setup of the ESP32 device on the Arduino Cloud IoT platform, which involves device registration and the acquisition of essential credentials. Subsequently, students create a virtual ”Thing” on the platform, linking it to the ESP32 and defining variables for data exchange, namely Gas_{ppm} and LED. Ensuring secure data transmission, students configure network credentials and incorporate the generated secret key.

The lab progresses to programming the ESP32, enabling it to process gas sensor data and manage the LED using digital control logic. Finally, students design a user-friendly dashboard within the XAMPP and Arduino Cloud IoT platform, integrating widgets for real-time sensor data visualization and remote LED control. Final outcomes of this experiment will be-

1. Proficient ESP32 setup for gas sensor monitoring.
2. Secure IoT data transmission using XAMPP and Arduino Cloud.
3. Get a basic understanding of how databases work using PHP.
4. How to prepare a simple webpage to show sensor data.
5. Effective use of virtual ”Thing” for data exchange in Arduino Cloud.
6. Competent programming for sensor data processing and LED control. write this in latex code
7. User-friendly IoT dashboard design for real-time data visualization.

Apparatus:

- ESP32 , USB cable ,Breadboard , DHT11 , LED ,Buzzer , Push Button ,Resistor: Typically 10k ohm

Theory:

What is Internet of Things (IoT)?

The Internet of Things, or IoT, is a system where physical devices, machines, and sensors are connected together through the internet. These devices can collect data from their surroundings and share this information with each other or with a central computer.

Think of it this way: imagine your home devices (like a thermometer, fan, or light bulb) all talking to each other and to your smartphone through the internet. That is IoT. In this experiment, we use an ESP32 microcontroller as the “smart device” that collects temperature and humidity data from sensors and sends this information to a server where it can be stored and viewed.

ESP32 Microcontroller

What is a Microcontroller?

A microcontroller is a small computer on a single chip. It is designed to control devices and perform specific tasks. Unlike your laptop or smartphone which can do many things, a microcontroller is built to do one or a few things very well. In this experiment, the ESP32 is our microcontroller.

Why Use ESP32?

The ESP32 is chosen for this experiment because it has several important features:

- **Built-in WiFi:** The ESP32 can connect to the internet without needing extra equipment. This allows our device to send data to a remote server.
- **Multiple Sensor Connections:** It has many pins and ports that can be connected to different sensors like temperature sensors and gas sensors.
- **Memory:** The ESP32 has enough memory to store programs and data temporarily while it is working.
- **Processing Power:** It can run fast enough to read sensor data and process it in real-time.
- **Security Features:** It supports encryption to keep our data safe when sending it over the internet.

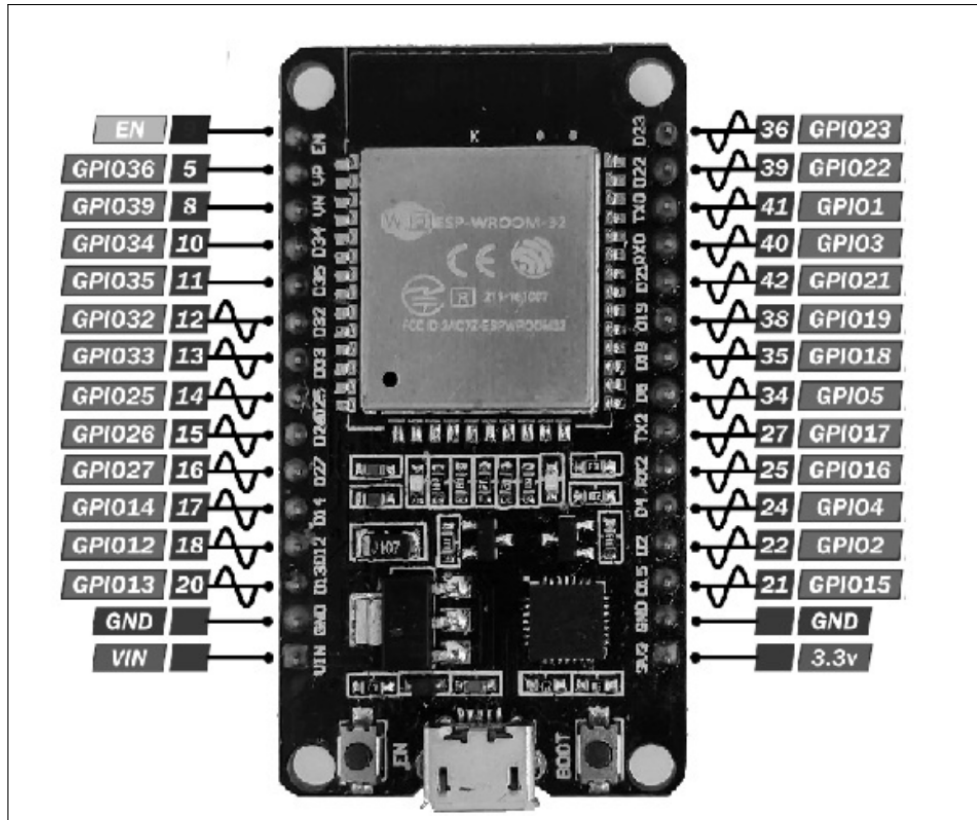


Figure 1: ESP32 Microcontroller Board

Sensors Used in This Experiment

1. DHT11 Temperature and Humidity Sensor

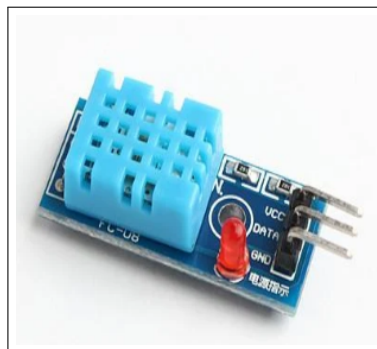


Figure 2: DHT11 Temperature and Humidity Sensor

The DHT11 is a sensor that measures two things at the same time: temperature and humidity (the amount of moisture in the air).

How it works: The sensor has a special material inside that changes its electrical properties when the temperature or humidity changes. The ESP32 can read these changes and calculate the actual temperature and humidity values.

Why we use it: This sensor is cheap, reliable, and easy to use with the ESP32.

Important points:

- It can measure temperature from 0°C to 50°C
- It can measure humidity from 20% to 80%
- It needs only three wires to connect to the ESP32: Power, Ground, and Data

2. MQ-2 Gas Sensor

The MQ-2 is a sensor that detects smoke and harmful gases in the air.

How it works: Inside the MQ-2, there is a special material that changes its resistance (electrical property) when it comes into contact with smoke or gas. The more smoke or gas present, the more the resistance changes. The ESP32 can read this change and tell us if there is smoke or gas nearby.

Why we use it: This sensor is useful for safety purposes. It can alert us if there is smoke (fire hazard) or dangerous gas in a room.

Important points:

- It can detect various gases like smoke, methane, and other harmful gases
- It gives both a digital output (yes/no) and an analog output (strength of detection)
- It takes a few minutes to warm up when powered on

XAMPP Server and Database

What is a Server?

A server is a computer that stores information and responds to requests. When you ask for something (like sensor data), the server finds it and sends it back to you. In this experiment, XAMPP creates a small server on your laptop so you can practice without using the real internet.

What is XAMPP?

XAMPP is a package that combines four important tools needed to run a website or web application on your computer:

- **Apache:** This is the web server software. It listens for requests from the ESP32 (or your web browser) and sends back the requested information.
- **MySQL (or MariaDB):** This is the database system. It stores all the sensor data (temperature, humidity, gas levels) that we receive from the ESP32 in an organized way, like a filing cabinet.

- **PHP:** This is a programming language that runs on the server. It acts as a bridge between the ESP32 and the database. When the ESP32 sends data, PHP receives it and stores it in the database.
- **Perl:** This is another programming language, though we don't use it much in this experiment.

How XAMPP Works Together

Think of XAMPP like a restaurant:

1. **Apache** is the waiter. It takes your order (request) and brings you food (information).
2. **PHP** is the chef. It receives your order and prepares it according to your needs.
3. **MySQL** is the storage room. It keeps all the ingredients (data) organized so the chef can use them quickly.

When the ESP32 sends temperature data, here's what happens:

- Apache receives the request from the ESP32
- PHP processes the data and prepares it
- MySQL stores the data in the database
- Apache sends back a confirmation message

What is a Database?

A database is an organized collection of data (information). Instead of writing things down on paper, a database stores everything digitally in a structured way, like a spreadsheet.

Benefits of a database:

- Easy to search and find information
- Can store large amounts of data
- Prevents duplicate information
- Allows multiple users to access data at the same time
- Can keep track of when data was added (timestamp)

In our experiment, the database has a table with columns for:

- **ID:** A unique number for each record
- **Temperature:** The temperature value from DHT11 sensor
- **Humidity:** The humidity value from DHT11 sensor
- **DateTime:** The date and time when the data was recorded

Communication Protocol: HTTP

What is HTTP?

HTTP (HyperText Transfer Protocol) is a language that computers use to talk to each other over the internet. It is like a set of rules that both sides must follow so they understand each other.

How HTTP works:

1. The ESP32 prepares the data (temperature and humidity values)
2. ESP32 sends this data to the server using HTTP
3. The server receives the data
4. The server processes the data and sends back a response
5. ESP32 receives the response and knows if the operation was successful

HTTP Request and Response

*Request: When the ESP32 wants to send data, it creates an HTTP request that looks something like this:

```
POST /dht11_project/testdata.php HTTP/1.1
Host: 192.168.0.111
temperature=25&humidity=65
```

This means: “Please receive this data (temperature=25, humidity=65) at the file testdata.php on this computer.”

Response: The server sends back a response:

```
HTTP/1.1 200 OK
Content-Type: text/plain
```

New record created successfully

This means: “Success! I received your data and stored it in the database.”

WiFi Connectivity

Why WiFi?

WiFi allows the ESP32 to connect to the internet without using cables. This makes it portable and flexible for IoT applications.

How ESP32 connects to WiFi:

1. The ESP32 scans for available WiFi networks (like your home WiFi)
2. You provide the network name (SSID) and password
3. ESP32 connects to the WiFi router
4. Once connected, ESP32 can communicate with any device on the network, including the XAMPP server

IP Address

Every device on a network has a unique address called an IP address, like a postal address for computers. The ESP32 needs to know the IP address of the server to send data to it.

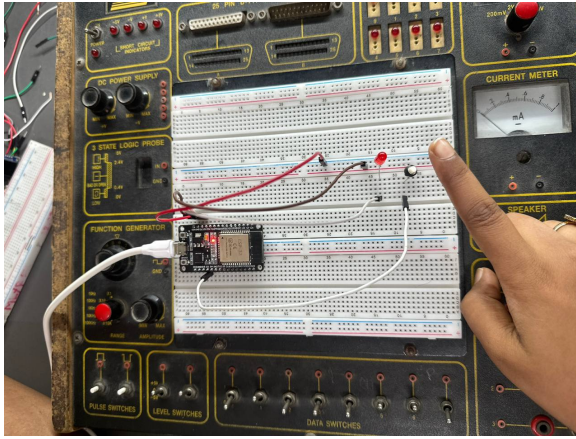
Example: If your computer's IP address is 192.168.0.111, then the ESP32 sends data to:

`http://192.168.0.111/dht11_project/testdata.php`

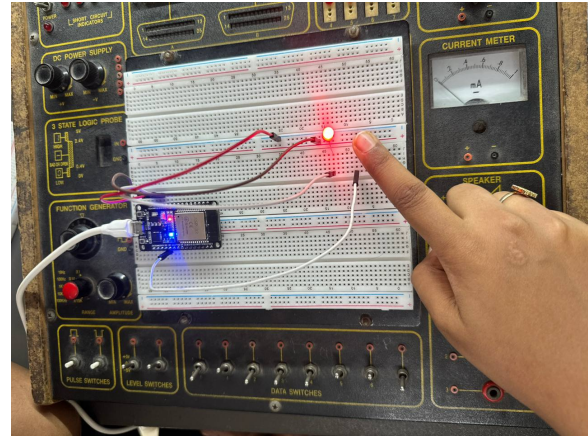
Solution of the mentioned problems:

1. Connect a push button and an LED to ESP32 pins. Write a program where the LED turns on when the button is pressed and turns off when it's released.

- Connected the push button to pin GPIO 2 as an input and the LED to pin GPIO 13 as an output.
- Programmed the ESP32 to continuously read the state of the push button in the main loop.
- Turned the LED ON when the push button was pressed (HIGH state).
- Turned the LED OFF when the push button was released (LOW state).
- Verified the operation on hardware as shown in Figure 3 and listed the pinouts in Table 1.



(a) Button Released (LED OFF)



(b) Button Pressed (LED ON)

Figure 3: ESP32 LED Control Circuit via Push Button Input - OFF and ON States

Table 1: ESP32 Hardware Pin Connections for Push Button and LED Circuit

Component	Component Pin	ESP32 Pin	Function / Description
Push Button	Terminal 1	GPIO 2	Digital Input (Reads HIGH-/LOW button state)
	Terminal 2	3.3V / GND	Pull-down / Pull-up Reference
LED	Anode (+)	GPIO 13	Digital Output (Drives LED state)
	Cathode (-)	GND	Ground Reference (via Resistor)

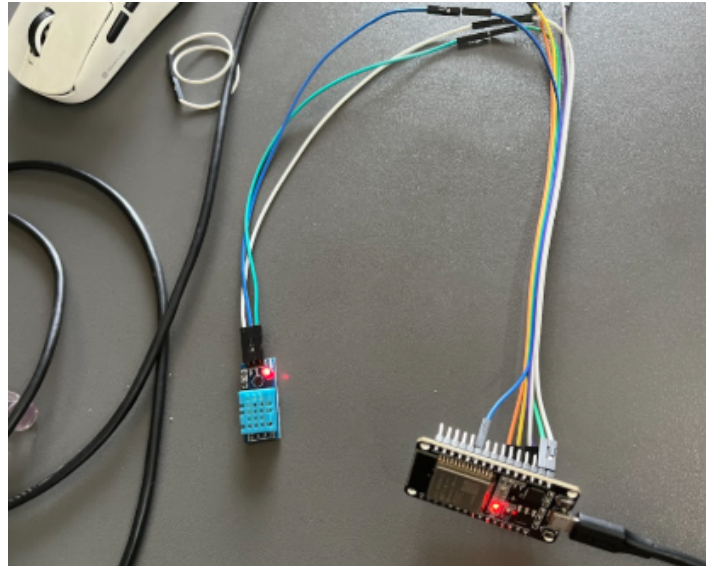
```

1 int estadobotao = 0;
2
3 void setup()
4 {
5     pinMode(2, INPUT);
6     pinMode(13, OUTPUT);
7 }
8
9 void loop()
10 {
11     estadobotao = digitalRead(2); // ler estadobotao
12
13     if (estadobotao == HIGH)
14     {
15         digitalWrite(13, HIGH);
16     } else {
17         digitalWrite(13, LOW);
18     }
19
20     delay(15); // somente para melhorar a simulacao
21 }

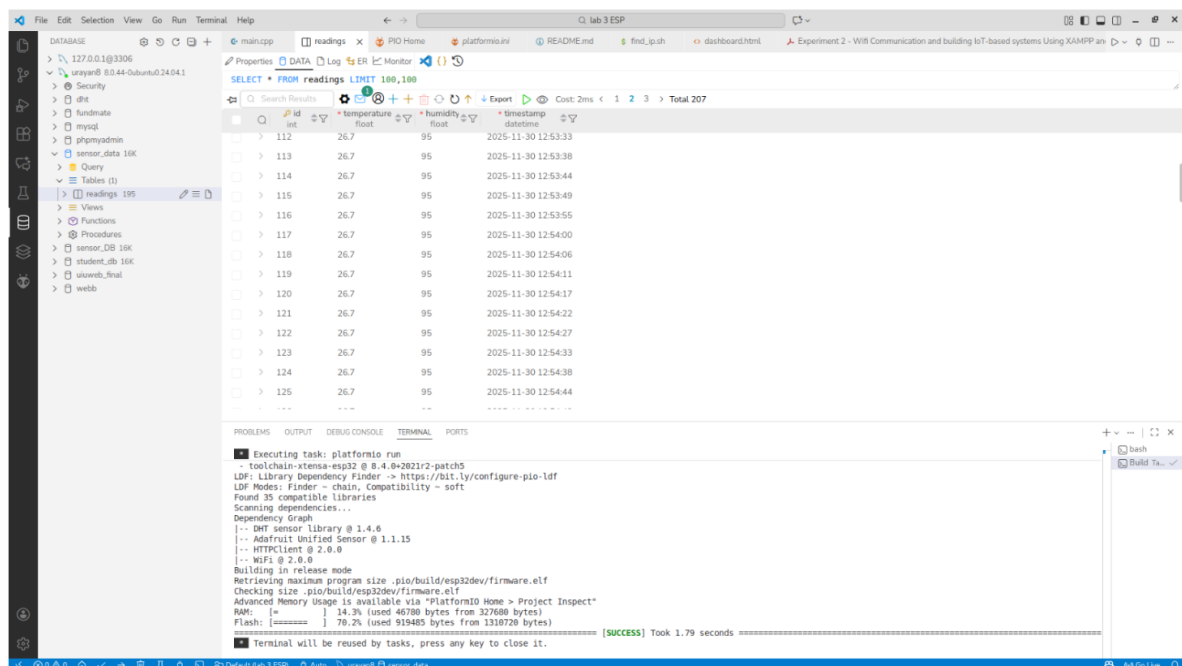
```

Listing 1: ESP32 Push Button and LED Control Code

2. Connect a DHT11 sensor to an ESP32 pin & read temperature and humidity data. Display this data on the console. Additionally, save this sensor data into an Excel file.



(a) Connection of DHT11 to an ESP32



(b) Database View showing Stored Sensor Data Readings

Figure 4: DHT11 Sensor Circuit Setup and Output Logs

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

* Executing task: platformio run
- toolchain-xtensa-esp32 @ 8.4.0+2021r2-patch5
LDF: Library Dependency Finder -> https://bit.ly/configure-pio-ldf
LDF Modes: Finder ~ chain, Compatibility ~ soft
Found 35 compatible libraries
Scanning dependencies...
Dependency Graph
|-- DHT sensor library @ 1.4.6
|-- Adafruit Unified Sensor @ 1.1.15
|-- HTTPClient @ 2.0.0
|-- WiFi @ 2.0.0
Building in release mode
Retrieving maximum program size .pio/build/esp32dev/firmware.elf
Checking size .pio/build/esp32dev/firmware.elf
Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
RAM: [=====] 14.3% (used 46780 bytes from 327680 bytes)
Flash: [=====] 70.2% (used 919485 bytes from 1310720 bytes)
===== [SUCCESS] Took 1.79 seconds =====

* Terminal will be reused by tasks, press any key to close it.

```

Figure 5: PlatformIO Terminal Output Showing Successful Compilation and Flash Memory Usage for ESP32 Firmware

Table 2: ESP32 Hardware Pin Mapping for DHT11 Sensor

Component	Module Pin	ESP32 Pin	Function / Description
DHT11 Sensor	VCC	3.3V	Power Supply Reference
	DATA	GPIO 19	Single-Wire Digital Signal Input
	GND	GND	Ground Reference Connection

- **1. Sensor Reading:** The ESP32 reads real-time temperature and humidity data from the DHT11 sensor connected to GPIO 19.
- **2. Serial Monitoring:** The microcontroller formats the captured sensor values and prints them to the local Serial Monitor.
- **3. Network Connection:** The ESP32 connects to the Wi-Fi network and constructs an HTTP GET request containing the sensor payload.
- **4. HTTP Transmission:** The ESP32 transmits the data request wirelessly to the Apache web server running on the Linux host every 5 seconds.
- **5. Server Processing & Storage:** The PHP script executes, writing the incoming measurements into a MySQL database with timestamps and appending them to a backup log file].

0.1 PlatformIO Configuration (platformio.ini)

```

1 [env:esp32dev]
2 platform = espressif32
3 board = esp32dev
4 framework = arduino

```

```

5 monitor_speed = 115200
6 lib_deps =
7     adafruit/DHT sensor library@^1.4.6
8     adafruit/Adafruit Unified Sensor@^1.1.14

```

0.2 ESP32 Firmware Code (src/main.cpp)

```

1 #include <Arduino.h>
2 #include <DHT.h>
3 #include <WiFi.h>
4 #include <HTTPClient.h>
5
6 // WiFi credentials
7 const char* ssid = "UIU-STUDENT";
8 const char* password = "12345678";
9
10 // Server URL
11 const char* serverName = "http://10.15.5.249/sensor_data.php";
12
13 // DHT11 Sensor configuration
14 #define DHTPIN 19
15 #define DHTTYPE DHT11
16 DHT dht11(DHTPIN, DHTTYPE);
17
18 float temperature = 0;
19 float humidity = 0;
20
21 void Load_DHT11_Data() {
22     humidity = dht11.readHumidity();
23     temperature = dht11.readTemperature();
24 }
25
26 void connectToWiFi() {
27     Serial.print("Connecting to ");
28     Serial.println(ssid);
29
30     WiFi.begin(ssid, password);
31
32     while (WiFi.status() != WL_CONNECTED) {
33         delay(500);
34         Serial.print(".");

```

```

35     }
36
37     Serial.println("\nWiFi connected!");
38     Serial.print("IP address: ");
39     Serial.println(WiFi.localIP());
40 }
41
42 void sendDataToServer() {
43     HTTPClient http;
44
45     String url = String(serverName) + "?temperature=" +
46                 String(temperature) + "&humidity=" + String(
47                 humidity);
48
49     Serial.print("Sending data to: ");
50     Serial.println(url);
51
52     http.setTimeout(5000);
53     http.begin(url.c_str());
54
55     int httpResponseCode = http.GET();
56
57     if (httpResponseCode > 0) {
58         Serial.print("HTTP Response code: ");
59         Serial.println(httpResponseCode);
60         String response = http.getString();
61         Serial.println("    Server response: " + response);
62     } else {
63         Serial.print("Error code: ");
64         Serial.println(httpResponseCode);
65     }
66
67     http.end();
68 }
69
70 void setup() {
71     Serial.begin(115200);
72     Serial.println("\nESP32 DHT11 WiFi System Starting...");
73
74     dht11.begin();
75     Serial.println("Waiting for DHT11 sensor to stabilize...");

```

```

75     delay(2000);
76
77     connectToWiFi();
78     Serial.println("\n=== System Ready ===");
79 }
80
81 void loop() {
82     if (WiFi.status() == WL_CONNECTED) {
83         Load_DHT11_Data();
84
85         Serial.print("Temperature: ");
86         Serial.print(temperature);
87         Serial.println(" C ");
88         Serial.print("Humidity: ");
89         Serial.print(humidity);
90         Serial.println(" %");
91
92         sendDataToServer();
93     } else {
94         Serial.println("WiFi Disconnected! Reconnecting...");
95         connectToWiFi();
96     }
97
98     delay(5000); // Send data every 5 seconds
99 }

```

0.3 Server-Side Database Setup (setup_database.sql)

```

1  -- Access MySQL via CLI: sudo mysql -u root
2
3  CREATE DATABASE IF NOT EXISTS sensor_data;
4  USE sensor_data;
5
6  CREATE TABLE IF NOT EXISTS readings (
7      id INT AUTO_INCREMENT PRIMARY KEY,
8      temperature FLOAT NOT NULL,
9      humidity FLOAT NOT NULL,
10     timestamp DATETIME NOT NULL
11 );
12
13 -- Create dedicated database user for ESP32

```

```

14 CREATE USER IF NOT EXISTS 'esp32'@'localhost' IDENTIFIED BY '
    esp32pass';
15 GRANT ALL PRIVILEGES ON sensor_data.* TO 'esp32'@'localhost';
16 FLUSH PRIVILEGES;

```

0.4 Server-Side Backend Script (/var/www/html/sensor_data.php)

```

1 <?php
2 header('Content-Type: text/plain');
3
4 // Get data from ESP32 GET parameters
5 $temperature = isset($_GET['temperature']) ? floatval($_GET['
    temperature']) : 0;
6 $humidity = isset($_GET['humidity']) ? floatval($_GET['humidity'
    ]) : 0;
7
8 // Database configuration
9 $servername = "localhost";
10 $username = "esp32";
11 $password = "esp32pass";
12 $dbname = "sensor_data";
13
14 // Create connection
15 $conn = new mysqli($servername, $username, $password, $dbname);
16
17 // Check connection
18 if ($conn->connect_error) {
19     die("Connection failed: " . $conn->connect_error);
20 }
21
22 // Insert data using prepared statement
23 $stmt = $conn->prepare("INSERT INTO readings (temperature,
    humidity, timestamp) VALUES (?, ?, NOW())");
24 $stmt->bind_param("dd", $temperature, $humidity);
25
26 if ($stmt->execute()) {
27     echo "OK - Data saved! Temperature: $temperature C, Humidity:
    $humidity %";
28 } else {
29     echo "Error: " . $stmt->error;
30 }

```

```

31
32 $stmt->close();
33 $conn->close();
34
35 // Log to text file for backup
36 $logFile = 'sensor_log.txt';
37 $timestamp = date('Y-m-d H:i:s');
38 $logData = "[$timestamp] Temperature: $temperature C, Humidity:
    $humidity %\n";
39 file_put_contents($logFile, $logData, FILE_APPEND);
40 ?>

```

3. Display DHT11 sensor data on an I2 COLED display

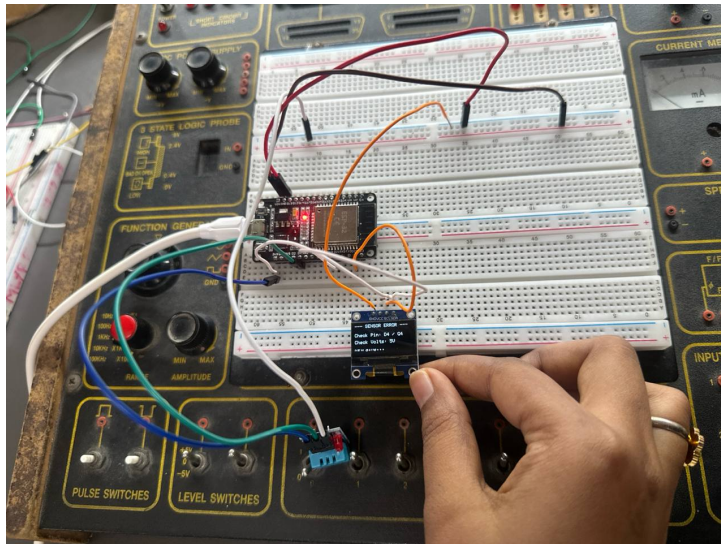


Figure 6: Hardware Implementation of ESP32 Interfaced with DHT11 Sensor and OLED Display on Trainer Board

```

1 #include <SPI.h>
2 #include <Wire.h>
3 #include <Adafruit_GFX.h>
4 #include <Adafruit_SSD1306.h>
5 #include <DHT.h>
6
7 // OLED Display Configuration
8 #define SCREEN_WIDTH 128
9 #define SCREEN_HEIGHT 64
10 #define OLED_RESET -1
11 #define SCREEN_ADDRESS 0x3C

```

```

12 Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire,
    OLED_RESET);
13
14 // DHT11 Sensor Configuration
15 #define DHTPIN 4           // Physical pin D4 on ESP32
16 #define DHTTYPE DHT11
17
18 // The '11' parameter adds an extra timing buffer for high-speed
    microcontrollers like the ESP32
19 DHT dht(DHTPIN, DHTTYPE, 11);
20
21 void setup() {
22     Serial.begin(115200);
23
24     // Initialize DHT11 with a software pull-up to ensure stable
        signals
25     pinMode(DHTPIN, INPUT_PULLUP);
26     dht.begin();
27
28     // Initialize OLED Display
29     if(!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
30         Serial.println(F("SSD1306 allocation failed"));
31         for(;;);
32     }
33
34     // Display initial loading sequence
35     display.clearDisplay();
36     display.setTextSize(1);
37     display.setTextColor(SSD1306_WHITE);
38     display.setCursor(0, 10);
39     display.println(F("ESP32 Weather System"));
40     display.setCursor(0, 30);
41     display.println(F("Connecting Sensor..."));
42     display.display();
43     delay(3000); // 3-second delay gives the DHT11 module proper
        time to warm up
44 }
45
46 void loop() {
47     // DHT11 updates slowly; a 3-second cycle prevents
        communication timeouts

```

```

48     delay(3000);
49
50     float humidity = dht.readHumidity();
51     float temperature = dht.readTemperature(); // Celsius
52
53     // Error check sequence
54     if (isnan(humidity) || isnan(temperature)) {
55         Serial.println("Failed to read from DHT sensor!");
56         display.clearDisplay();
57         display.setTextSize(1);
58         display.setCursor(0, 0);
59         display.println(F("--- SENSOR ERROR ---"));
60         display.setCursor(0, 20);
61         display.println(F("Check Pin: D4 / G4"));
62         display.setCursor(0, 35);
63         display.println(F("Check Volts: 5V"));
64         display.setCursor(0, 50);
65         display.println(F("Retrying..."));
66         display.display();
67         return;
68     }
69
70     // Output telemetry data to Serial Monitor
71     Serial.print("Humidity: ");
72     Serial.print(humidity, 1);
73     Serial.print("% Temperature: ");
74     Serial.print(temperature, 1);
75     Serial.println("C");
76
77     // Output telemetry data to OLED Screen
78     display.clearDisplay();
79
80     // Title Bar Layout
81     display.setTextSize(1);
82     display.setCursor(0, 0);
83     display.println(F("--- WEATHER DATA ---"));
84
85     // Temperature Layout Row
86     display.setTextSize(2);
87     display.setCursor(0, 20);
88     display.print(F("T: "));

```

```

89  display.print(temperature, 1);
90  display.setTextSize(1);
91  display.cp437(true);
92  display.write(167);           // Render standard degree symbol
93  display.setTextSize(2);
94  display.println(F("C"));
95
96  // Humidity Layout Row
97  display.setCursor(0, 45);
98  display.print(F("H: "));
99  display.print(humidity, 1);
100 display.println(F(" %"));
101
102 display.display();
103 }

```

Listing 2: ESP32 Firmware Code for DHT11 Sensor and OLED Display

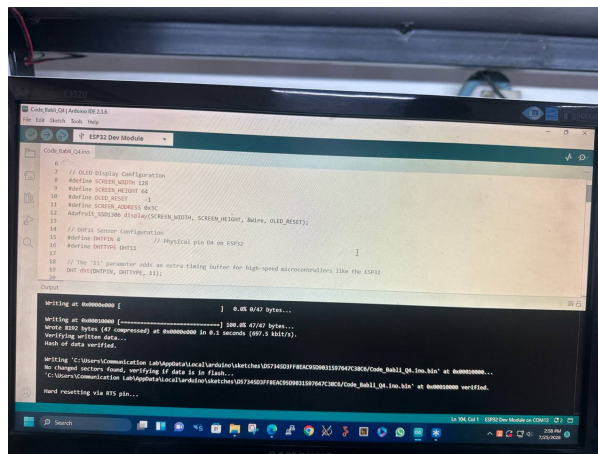


Figure 7: Arduino IDE Interface Demonstrating Successful Code Compilation and Firmware Flashing to the ESP32

- Connected the ESP32 to an SSD1306 OLED display via I^2C and a DHT11 sensor to pin GPIO 4.
- Programmed the ESP32 in Arduino IDE using the Adafruit SSD1306, GFX, and DHT libraries.
- Compiled and flashed the firmware binary to the ESP32 Dev Module over serial port COM13.
- Monitored real-time temperature and humidity data on the OLED display and Serial Monitor.

Table 3: ESP32 Hardware Pin Mapping for OLED Display and DHT11 Sensor

Component	Module Pin	ESP32 Pin	Function / Description
SSD1306 OLED	VCC	3.3V / 5V	Power Supply
	GND	GND	Ground Reference
	SDA	GPIO 21	I^2C Data Line
	SCL	GPIO 22	I^2C Clock Line
DHT11 Sensor	VCC	5V	Power Supply
	GND	GND	Ground Reference
	DATA	GPIO 4 (D4)	Single-Wire Digital Data Input

Discussion:

In these experiments, I first used a push button to turn an LED on and off. Then, I connected a DHT sensor to measure temperature and humidity, displayed the live values on the serial monitor, and saved the data to a database. Finally, I used the I2C communication method to display the live temperature and humidity values on an OLED screen, creating a simple embedded system with real-time sensor reading and display.